COURSE PROJECT · ACADEMIC DEFENSE EDITION

CodeGuardian

Deterministic Compiler-Based Static Taint Analysis Engine for Vulnerability Detection

Course Alignment: Compiler Design Theory (BCSE307L) & Lab (BCSE307P)

Department: School of Computer Science & Engineering (SCOPE)

Institution: Vellore Institute of Technology (VIT)

Academic Team: Member 1 (Compiler Core & Lowering) & Member 2 (Security Solver & UI)

Repository: `https://github.com/MayurRajeshKeni/codeguardian`

Date of Publication: October 2026

Complete System Engineering Handbook · Architectural Specification · Directory & Codebase Guide · Full Development Log (Sessions 1–7) · Comprehensive Viva Defense Q&A (Modules 1–7)

Table of Contents

1. Executive Summary & Project Overview	**Page 3**
1.1 The Core Problem: Why Static Taint Analysis?	Page 3
1.2 Mathematical Foundations: The Monotone Powerset Lattice	Page 4
1.3 The 6-Stage Compilation & Analysis Pipeline	Page 5
2. Complete Codebase & Directory Architecture	**Page 6**
2.1 Compiler Core Subsystem (C11, Flex, Bison)	Page 6
2.2 Security Engine Subsystem (Node.js, Kildall Solver)	Page 8
2.3 Interactive Dashboard Subsystem (React 18, Vite, Cytoscape.js)	Page 10
2.4 Benchmark Test Bench, Verification Scripts & Governance	Page 12
3. Full Engineering Development Log (Sessions 1 to 7)	**Page 13**
3.1 Session 1 (Day 1): Governance, Threat Model & Contracts	Page 13
3.2 Session 2 (Day 2): Lexer, LALR(1) Grammar & Conflict Elimination	Page 14
3.3 Session 3 (Day 3): AST Synthesis & Post-Order SDT to TAC	Page 15
3.4 Session 4 (Day 4): Dragon Book Leader Partitioning & CFG Synthesis	Page 16
3.5 Session 5 (Day 5): Kildall Worklist Fixpoint Solver & Path Tracer	Page 17
3.6 Session 6 (Day 6): Dual-Pane UI, Cytoscape Graph & SLA Benchmarking	Page 18
3.7 Session 7 (Day 7): UI Overhaul, Semantic Remediation, SARIF & Themes	Page 19
4. Academic Viva Defense Guide & Master Q&A	**Page 20**
4.1 Syllabus Alignment Matrix (Theory to Code Mapping)	Page 20
4.2 Deep Theory: DFA, LALR(1), Leaders, and Monotone Lattices	Page 21
4.3 Top 25 Viva Defense Questions & Comprehensive Answers	Page 22
5. Performance Benchmarks & Milestone Gate Acceptance	**Page 29**

1. Executive Summary & Project Overview

CodeGuardian is an end-to-end, deterministic compiler-based static analysis platform engineered to detect dataflow security vulnerabilities in source code before execution. Developed strictly within the theoretical and practical framework of university compiler design syllabi (**BCSE307L / BCSE307P**), the system integrates classical front-end language processing with modern data-flow lattice mathematics and an interactive visual intelligence dashboard.

> **Core Mission & Academic Objective**
>
> Demonstrate how every classical compiler phase—from Lexical DFA scanning and LALR(1) syntax analysis to Syntax-Directed Translation (SDT), Three-Address Code (TAC) generation, and Basic Block Leader Partitioning—directly serves as the mathematical prerequisite for industrial-grade static security analysis.

1.1 The Core Problem: Why Static Taint Analysis?

Modern software development suffers from critical injection vulnerabilities (such as SQL Injection - CWE-89, Command Injection - CWE-78, and Cross-Site Scripting - CWE-79). Traditional AST grep and Regular Expression scanners only match surface-level syntax strings (e.g. searching for calls to `execute_query()`). However, regex is completely blind to:

- **Control-Flow Divergence:** Regex cannot evaluate whether input was checked or sanitized along specific true/false branch execution paths.
- **Loop Accumulation:** Regex cannot track variables modified across repetitive loop cycles.
- **Variable Aliasing & Arithmetic Propagation:** If untrusted data is copied through multiple assignments (`x = input; y = x + 1; z = y;`), regex fails entirely.

The CodeGuardian Solution: By lowering source code into Three-Address Code (TAC) and constructing a formal Control Flow Graph (CFG), CodeGuardian executes **Kildall's Monotone Worklist Fixpoint Algorithm** over a mathematical powerset lattice. It proves whether an untrusted taint source can reach a sensitive sink along *any* feasible execution path without prior neutralization by an authorized sanitizer.

1.2 Mathematical Foundations: The Monotone Powerset Lattice

Static taint analysis in CodeGuardian is formally defined as a **Forward, May-Analysis** over the powerset semi-lattice:

```
Lattice Structure: (P(V), ⊆)
where:
  V = The finite set of program variable identifiers
  ⊥ = ∅ (Bottom: No variables tainted)
  ⊤ = V (Top: All variables tainted)
  ⊔ = ∪ (Join operator: Set union over incoming execution paths)
```

For every basic block \(B \), the input taint set \(IN[B] \) and output taint set \(OUT[B] \) are governed by the monotone transfer equations:

```
IN[B]  = ∪ { OUT[P] | P ∈ Pred[B] }
OUT[B] = GEN[B] ∪ (IN[B] \ KILL[B])
```

Transfer Component Definitions:

- **\(GEN[B] \):** The set of variables tainted within basic block \(B \). This occurs when a variable is directly assigned the result of an untrusted source call (e.g. `x = read_input()` or `p = get_param()`), or derived through arithmetic operations where any operand is already tainted.
- **\(KILL[B] \):** The set of variables cleansed within basic block \(B \). This occurs when a variable passes through an authorized sanitizer (e.g. `x = sanitize(x)` or `s = escape_sql(s)`), or is overwritten with a clean constant literal (e.g. `x = 0`).

> **Tarski's Fixed Point Theorem & Guaranteed Termination**
>
> Because the set of program variables \(V \) is finite, the powerset lattice \((\mathcal{P}(V), \subseteq) \) has a finite maximum height of \(|V| \). Because transfer functions are strictly monotonic (\(X \subseteq Y \implies f_B(X) \subseteq f_B(Y) \)), Kildall's iterative worklist solver is mathematically guaranteed to terminate at the unique **Least Fixed Point (LFP)** in at most \(\mathcal{O}(|V| imes |E|) \) operations, with zero risk of non-termination even across complex cyclic loop back-edges.

1.3 The 6-Stage Compilation & Analysis Pipeline

CodeGuardian processes source code through six strongly decoupled, contractually frozen stages:

Stage	Subsystem	Primary Technologies	Output Artifact / Contract
1. Lexical Analysis	Compiler Core	GNU Flex (2.6.4), C11 DFA State Machine	Deterministic Token Stream with line/column coordinates
2. Syntax Analysis	Compiler Core	GNU Bison (3.8.2), LALR(1) Pushdown Automaton	Concrete Parse Tree (0 shift/reduce conflicts)
3. AST Synthesis	Compiler Core	C11 In-Memory Tree Construct & Destructors	Hierarchical Abstract Syntax Tree (`AST.json`)
4. TAC & CFG Partitioning	Compiler Core	Syntax-Directed Translation, Dragon Book 3-Rule Leaders	Control Flow Graph & Quadruples (`CFG.json`)
5. Kildall Fixpoint Solver	Security Engine	Node.js, Monotone Worklist over \((\mathcal{P}(V), \subseteq) \)	Lattice States & Path Audit (`Audit.json`)
6. Visual Intelligence UI	Dashboard	React 18, Vite, Cytoscape.js, Dagre Layout	Interactive Graph, Auto-Remediation & SARIF Export

2. Complete Codebase & Directory Architecture

The CodeGuardian repository is strictly divided across clear architectural boundaries to separate compiler theory (C11) from analysis intelligence (Node.js) and frontend visualization (React/Vite). Below is the comprehensive directory blueprint:

```
CD_CodeGuardian/
├── Makefile                          # Root build, clean, test, and benchmark orchestrat
├── README.md                         # Master architectural overview & documentation hub
├── RUNBOOK.md                        # Cross-platform execution & operational runbook
├── mock_CFG.json                     # Milestone 1 frozen CFG contract fixture
├── mock_Audit.json                   # Milestone 1 frozen Audit contract fixture
│
├── compiler_core/                    # [Member 1] Native C11 Front-End & Lowering
│   ├── Makefile                      # Toolchain build driver (Flex, Bison, GCC)
│   ├── CMakeLists.txt                # CMake build configuration
│   ├── BNF_Grammar.md                # Formal Backus-Naur Form language grammar
│   ├── src/                          # C11 Implementation sources
│   │   ├── scanner.l                 # GNU Flex lexer definition (regex → DFA)
│   │   ├── parser.y                  # GNU Bison LALR(1) grammar definition
│   │   ├── tokens.h / tokens.c       # Token descriptors & string representation
│   │   ├── ast.h / ast.c             # In-memory AST node constructors & JSON emitter
│   │   ├── tac.h / tac.c             # Syntax-Directed Translation lowering AST to TAC
│   │   ├── cfg.h / cfg.c             # Dragon Book 3-rule leader partitioning & edges
│   │   ├── json_emit.h / json_emit.c # Contract serializers (CFG.json & AST.json)
│   │   └── main.c                    # CLI driver executable for frontend
│   └── tests/                        # Unit test suite verifying frontend lex/parse
│       ├── test_simple.c             # Arithmetic & scalar variable declarations
│       ├── test_branch.c             # Conditional if-else branching
│       ├── test_loop.c               # While loops & counter iterations
│       ├── test_sanitize.c           # Security primitive calls (sanitize, escape_sql)
│       ├── test_collision.c          # Lexical collision & keyword boundary checks
│       └── test_invalid.c            # Fail-closed syntax/lexer error tests
│
├── security_engine/                  # [Member 2] Security Intelligence & HTTP Gateway
│   ├── package.json                  # Dependencies (Ajv JSON schema validator)
│   ├── security_rules.json           # Source, Sanitizer, and Sink rule definitions
│   ├── schemas/                      # Frozen JSON schema definitions
│   │   ├── cfg.schema.json           # JSON Schema for incoming CFG contracts
│   │   └── audit.schema.json         # JSON Schema for outgoing Audit contracts
│   ├── src/                          # JavaScript data-flow analysis sources
│   │   ├── solver.js                 # Kildall monotone worklist fixpoint solver
│   │   ├── lattice.js                # Powerset lattice operations (join, subset, diff)
│   │   ├── transfer.js               # Instruction transfer functions (GEN/KILL sets)
│   │   ├── auditor.js                # Vulnerability detection & backtrack path tracer
│   │   ├── rules.js                  # Security rules loader & validator
│   │   ├── compile.js                # Subprocess bridge invoking compiler frontend
│   │   ├── server.js                 # HTTP REST analysis gateway (port 8787)
│   │   ├── cli.js                    # Headless CLI analyzer (stdin → Audit.json)
│   │   ├── validate.js               # Schema validation routines
│   │   └── errors.js                 # Structured diagnostic error classes
│   └── tests/                        # Automated test suites for solver & pipeline
│
├── dashboard/                        # [Member 2] Visual Intelligence Dashboard
│   ├── package.json                  # Dependencies (React 18, Cytoscape, Dagre, Vite)
│   ├── vite.config.js                # Vite build configuration & reverse proxy
│   ├── index.html                    # SPA HTML entry point
│   └── src/                          # UI Component sources
│       ├── main.jsx                  # React application bootstrap
│       ├── App.jsx                   # Master dual-pane UI, line jump, and controls
│       ├── Graph.jsx                 # Cytoscape.js canvas with theme-adaptive SVG nodes
│       ├── remediation.js            # Semantic AST/TAC auto-patch diff synthesizer
```

```
│       ├── sarif.js                    # OASIS SARIF 2.1.0 security report generator
│       ├── styles.css                  # Dual Dark/Light theme CSS design system
│       └── samples/                    # Pre-bundled Mini-C benchmark test cases
│
├── examples/                           # Canonical Benchmark Security Scenarios
│   ├── clean_flow.c                    # Clean flow: read_input → sanitize → query (0 vu
│   ├── sql_injection.c                 # Unsanitized flow to execute_query (CWE-89)
│   ├── command_injection.c             # Unsanitized branch to system_exec (CWE-78)
│   ├── xss.c                           # Unsanitized flow to render_output (CWE-79)
│   ├── complex_loop.c                  # Multi-iteration cyclic loop taint accumulator
│   └── master_showcase.c               # Combined multi-sink enterprise test case
│
├── DOC/                                # Academic Governance & Research Documentation
│   ├── PRD.md                          # Product Requirements Document & syllabus mapping
│   ├── Architecture.md                 # Formal system architecture & transfer math
│   ├── Rules.md                        # Strict technical guardrails & safety policies
│   ├── Phases.md                       # 5-phase delivery model & acceptance gates
│   ├── Design.md                       # Visual design tokens & graph styling spec
│   ├── Memory.md                       # Live task ledger & sprint state tracking
│   ├── devlog.md                       # Engineering development log (Sessions 1 to 7)
│   ├── Viva_Preparation_Guide.md       # Academic viva defense handbook (Modules 1-7)
│   └── Benchmark_Report.md             # Quantitative SLA latency verification report
│
└── scripts/                            # Cross-Platform Automation & Verification
    ├── run_tests.ps1                   # Compiler unit test runner (Windows)
    ├── benchmark.ps1                   # SLA benchmark stopwatch runner (Windows)
    ├── verify_all.ps1                  # Master 5-stage verification suite (Windows)
    └── verify_all.sh                   # Master 5-stage verification suite (POSIX)
```

2.1 Compiler Core Subsystem Details (C11, Flex, Bison)

The compiler core represents the classical front-end and middle-end of CodeGuardian. It is implemented in standard C11 without external dependencies beyond standard C runtime libraries:

- **scanner.l:** GNU Flex specification. Defines token patterns for Mini-C keywords (`int` , `bool` , `void` , `if` , `else` , `while` , `return`), operators, identifiers, and literals. Employs `YY_USER_ACTION` to track exact line, column, and character offsets. Implements the **Maximal Munch** rule to guarantee identifier tie-breaking.
- **parser.y:** GNU Bison LALR(1) specification. Implements stratified grammar rules for declarations, statements, and expressions. Resolves the classic dangling-else ambiguity using `%nonassoc LOWER_THAN_ELSE` and `%nonassoc TOK_ELSE` , yielding exactly **0 shift/reduce** and **0 reduce/reduce** conflicts.
- **ast.c / ast.h:** Dynamic in-memory Abstract Syntax Tree. Constructs typed nodes (statements, binary operations, function calls, variables, literals) and provides recursive post-order tree destruction to prevent memory leaks.
- **tac.c / tac.h:** Syntax-Directed Translation engine. Recursively lowers the AST into linear Three-Address Code quadruples $(ext{op}, ext{arg1}, ext{arg2}, ext{result}) $, generating temporary variables $t_0, t_1, \dots $ and jump labels $L_0, L_1, \dots $.

- **cfg.c / cfg.h:** Control Flow Graph generator. Implements the Dragon Book 3-Rule Leader Partitioning algorithm to divide linear TAC instructions into basic blocks. Dynamically constructs directed edges categorized as `TRUE_BRANCH`, `FALSE_BRANCH`, or `UNCONDITIONAL`.
- **json_emit.c / json_emit.h:** Serialization engine. Emits valid, formatted `CFG.json` and `AST.json` documents adhering strictly to frozen JSON schema contracts.
- **main.c:** CLI harness. Provides command-line options (`--lex`, `--parse`, `--ast`, `--tac`, `--cfg`) for standalone execution and piping.

2.2 Security Engine Subsystem Details (Node.js & Kildall Solver)

The security engine operates as the mathematical verification brain of CodeGuardian. It ingests `CFG.json` and calculates taint propagation:

- **solver.js:** Kildall Monotone Worklist Solver. Implements the forward iterative data-flow algorithm over the powerset lattice $(\mathcal{P}(V), \subseteq)$. Maintains a dynamic worklist queue of basic blocks, iteratively joining predecessor OUT sets into $IN[B]$ until reaching the unique Least Fixed Point (LFP).
- **lattice.js:** Pure mathematical lattice operations (set union $\cup$, set difference $\setminus$, subset test $\subseteq$, and equality comparison).
- **transfer.js:** Instruction-level transfer functions. Analyzes individual TAC quadruples to compute block-level $GEN[B]$ (taint ingestion and propagation) and $KILL[B]$ (sanitization calls and constant overwrites).
- **auditor.js:** Vulnerability detection engine. Inspects sensitive sink calls in each basic block against active IN sets. When a tainted argument triggers a sink, it executes a backward traversal along carrying edges to isolate the originating source and construct the complete propagation path $B_0 \circ \dots \circ B_{\text{sink}}$.
- **rules.js / security_rules.json:** Formal vulnerability rules binding CWE IDs (CWE-89, CWE-78, CWE-79) to authorized sources, sanitizers, and sensitive sinks.
- **compile.js:** Safe execution bridge that invokes the native C11 frontend binary via child processes and translates syntax errors into structured diagnostics.
- **server.js:** Zero-bloat HTTP REST API gateway running on port 8787. Exposes endpoints for static code analysis (`POST /api/analyze`), pre-bundled benchmark retrieval (`GET /api/examples`), and system health probes (`GET /health`).
- **cli.js:** Headless command-line tool allowing direct pipeline integration (`codeguardian-frontend code.c | node cli.js`).

2.3 Interactive Dashboard Subsystem Details (React 18 & Cytoscape.js)

The visual dashboard serves as the interactive cockpit for developers and course evaluators:

- **App.jsx:** Main React application container. Manages application state, code editor synchronization, line-jump flashing, split-pane dragging, collapsible sidebar toggles, and analysis execution guards.

- **Graph.jsx:** Cytoscape.js interactive canvas component. Implements the hierarchical Dagre layout, rendering basic blocks as dynamic, high-DPI SVG cards. Node colors dynamically reflect formal lattice states (Emerald Green for Clean, Cyan for Sanitized, Amber for Tainted, Crimson Red for Vulnerable). Supports auto-pan, smooth zoom, and animated glowing pulses on attack paths.
- **remediation.js:** Semantic AST/TAC auto-remediation engine. Identifies compromised variables, matches the corresponding authorized sanitizer, and synthesizes code diffs with an instant "Apply Fix & Re-Analyze" workflow.
- **sarif.js:** Enterprise report generator emitting OASIS SARIF 2.1.0 standard documents for GitHub Code Scanning and CI/CD ingestion.
- **styles.css:** Comprehensive design system featuring curated Dark and Light themes, smooth transitions, glassmorphic floating bars, and print-optimized media queries.

3. Full Engineering Development Log (Sessions 1 to 7)

3.1 Session 1 (Day 1): Governance, Threat Model & Mathematical Contracts

- **Key Achievements:** Drafted and locked core engineering specifications: `PRD.md`, `Architecture.md`, `Rules.md`, `Phases.md`, and `Design.md`. Frozen JSON schema definitions for `CFG.json` and `Audit.json` were established.
- **Engineering Significance:** Decoupled Member 1 (C11 Compiler Core) from Member 2 (Security Solver & Dashboard) from Day 1, allowing concurrent parallel development against mock contract fixtures.
- **Theoretical Rigor:** Formulated static taint analysis as a Forward, May-Analysis over the powerset semi-lattice $(\mathcal{P}(V), \subseteq)$, mathematically proving termination to the Least Fixed Point via Tarski's Theorem.

3.2 Session 2 (Day 2): Lexer, LALR(1) Grammar & Conflict Elimination

- **Key Achievements:** Implemented GNU Flex lexer (`scanner.l`) with coordinate tracking and verified zero lexical collisions. Built the GNU Bison LALR(1) grammar (`parser.y`) resolving the classic dangling-else ambiguity using `%nonassoc` precedence declarations, achieving **0 shift/reduce** and **0 reduce/reduce** conflicts.
- **Engineering Significance:** Ensured deterministic syntax parsing with zero backtracking, satisfying Milestone Acceptance Gate 1.

3.3 Session 3 (Day 3): AST Synthesis & Post-Order SDT to TAC

- **Key Achievements:** Implemented in-memory AST node allocators and post-order recursive tree free functions in `ast.c`. Built the Syntax-Directed Translation engine in `tac.c`, lowering nested expressions and statements into linear Three-Address Code quadruples with explicit temporaries and jump labels.
- **Engineering Significance:** Linearized nested tree expressions into atomic mathematical operations, a prerequisite for identifying basic block boundaries.

3.4 Session 4 (Day 4): Dragon Book Leader Partitioning & CFG Synthesis

- **Key Achievements:** Implemented the canonical Dragon Book 3-rule leader partitioning algorithm in `cfg.c`. Wired basic blocks with typed directed edges (`TRUE_BRANCH`, `FALSE_BRANCH`, `UNCONDITIONAL`) and built JSON serialization in `json_emit.c`, satisfying Milestone Acceptance Gate 2.

- **Engineering Significance:** Completed the C11 compiler frontend, delivering fully validated `CFG.json` output across all benchmark scenarios.

3.5 Session 5 (Day 5): Kildall Worklist Fixpoint Solver & Path Tracer

- **Key Achievements:** Built the Kildall monotone worklist solver in `solver.js`. Implemented mathematical transfer functions for \(GEN \) and \(KILL \) sets in `transfer.js`. Built the backwards path tracer in `auditor.js` and created the HTTP REST gateway in `server.js`.
- **Engineering Significance:** Delivered mathematically verified dataflow taint solving, verifying convergence across cyclic loop back-edges across 62 automated unit tests.

3.6 Session 6 (Day 6): Dual-Pane UI, Cytoscape Graph & SLA Benchmarking

- **Key Achievements:** Initialized React 18 / Vite dashboard with Cytoscape.js Dagre hierarchical canvas. Implemented dynamic attack path highlighting, TAC block detail drawer, and latency benchmarking suite (`benchmark.ps1`).
- **Engineering Significance:** Proved that the full end-to-end analysis pipeline executes in ~200 ms, beating the PRD NFR-02 SLA (< 300 ms) with over 100 ms of safety headroom.

3.7 Session 7 (Day 7): UI Overhaul, Semantic Remediation, SARIF & Themes

- **Key Achievements:**
  - Synchronized line flash jump animation from finding cards to source editor.
  - Attack path auto-pan and smooth zoom with pulsing glowing nodes.
  - Semantic auto-remediation patch engine with visual Diff modal and "Apply Fix & Re-Analyze".
  - OASIS SARIF 2.1.0 security report generator for CI/CD pipelines.
  - Full Dark and Light theme system with persistent toggle and adaptive SVG graph cards.
  - Calmed card design eliminating loud red borders, plus controlled manual analysis execution.
- **Engineering Significance:** Transformed CodeGuardian from an academic demonstration into an enterprise-grade, turnkey developer static analysis platform.

4. Academic Viva Defense Guide & Master Q&A

4.1 Syllabus Alignment Matrix (Theory to Code Mapping)

CodeGuardian was specifically architected to demonstrate direct, tangible mastery of all core syllabus modules in Compiler Design (**BCSE307L / BCSE307P**):

Syllabus Module	Theoretical Concept	Implementation File	Key Functions / Constructs
Module 1: Lexical Analysis	Regular Expressions, DFA Minimization, Maximal Munch, Buffer Coordinates	`compiler_core/src/scanner.l`	`yylex()`, `update_loc()`, `YY_USER_ACTION`
Module 2: Syntax Analysis	Context-Free Grammars, LALR(1) Parsing Tables, Shift/Reduce Disambiguation	`compiler_core/src/parser.y`	`yyparse()`, `%nonassoc LOWER_THAN_ELSE`, `%nonassoc TOK_ELSE`
Module 3: Abstract Syntax Trees	Hierarchical Syntax Trees, Semantic Actions, Memory Management	`compiler_core/src/ast.c`	`ast_create_node()`, `ast_add_child()`, `ast_free()`
Module 4: Syntax-Directed Translation	Post-order SDT, Intermediate Code Generation (ICG), 3-Address Code (TAC)	`compiler_core/src/tac.c`	`tac_lower_ast()`, `lower_expr()`, Temporary generator (\(t_i \))
Module 5: Basic Blocks & CFG	Dragon Book 3-Rule Leader Partitioning, Directed Control Flow Edges	`compiler_core/src/cfg.c`	`cfg_build_from_tac()`, `cfg_add_edge()`, `json_emit_cfg()`
Module 6: Data-Flow Analysis	Kildall Worklist Algorithm, Powerset Lattice, Least Fixed Point (LFP)	`security_engine/src/solver.js`	`solveLattice()`, `computeBlockGenKill()`, Worklist Queue
Module 7: Application	Static Taint Analysis,	`security_engine/src/auditor.js`	`auditBlock()`, `traceVulnerabilityPath()`,

Syllabus Module	Theoretical Concept	Implementation File	Key Functions / Constructs
Security	Source-Sanitizer-Sink, CWE Taxonomy (CWE-89, 78, 79)		`Audit.json`

4.2 Top 25 Viva Defense Questions & Comprehensive Answers

Q1 **What fundamentally distinguishes CodeGuardian from simple regex or AST pattern matching tools?**

Answer: Regex and AST grep tools perform purely syntactic pattern searches (e.g. detecting the string `execute_query`). They have zero understanding of control-flow topology, branching, variable sanitization, or loop accumulators. In contrast, CodeGuardian lowers source code into a Control Flow Graph and runs a monotone data-flow lattice solver over all execution paths. It only flags a vulnerability if an untrusted source can propagate to a sink without passing through an authorized sanitizer along that specific path, completely avoiding false alarms when sanitizers are present.

Q2 **Why is static taint analysis formulated as a "Forward, May-Analysis"?**

Answer: It is **Forward** because taint data flows in the direction of execution (from source statements at entry points forward toward sink calls). It is a **May-Analysis** because a variable is considered tainted at the entrance of a basic block if it can be tainted along *any* incoming execution path. Mathematically, incoming states are combined using the union join operator (\(igcup \)), reflecting worst-case security reachability.

Q3 **How does your LALR(1) parser resolve the classical dangling-else ambiguity with 0 conflicts?**

Answer: In `compiler_core/src/parser.y`, we declare two precedence tokens: `%nonassoc LOWER_THAN_ELSE` and `%nonassoc TOK_ELSE`. In the grammar rule, the single `if` branch is annotated with `%prec LOWER_THAN_ELSE`. When the parser encounters an `else` token, `TOK_ELSE` has higher precedence than `LOWER_THAN_ELSE`. This forces Bison's LALR(1) state machine to **shift** the `else` token onto the stack rather than reducing, binding the `else` to the innermost `if`. This completely eliminates shift/reduce conflicts (0 conflicts).

Q4 **State the Dragon Book 3-rule algorithm for partitioning basic block leaders.**

Answer:

1. **Rule 1:** The first Three-Address Code instruction in the intermediate representation is a leader.
2. **Rule 2:** Any instruction that is the target of a conditional or unconditional jump (e.g. labels targeted by `IF_FALSE` or `GOTO`) is a leader.
3. **Rule 3:** Any instruction that immediately follows a jump or return instruction is a leader.

A basic block consists of a leader and all consecutive instructions up to, but not including, the next leader.

Q5 **What is Three-Address Code (TAC), and why is it necessary before CFG partitioning?**

Answer: Three-Address Code is an intermediate representation where every instruction has at most one operator and at most three address operands. It linearizes hierarchical, deeply nested AST expressions into explicit, sequential quadruples with compiler-generated temporaries ($t_0, t_1, \dots$) and jump labels ($L_0, L_1, \dots$). Without linearizing expressions into atomic steps, basic block leader boundaries and data-flow transfer equations cannot be computed.

Q6 **What mathematical property guarantees that Kildall's worklist algorithm terminates on cyclic loops?**

Answer: Monotonicity over a finite lattice. The carrier set is the powerset of program variables $\mathcal{P}(V)$, which has finite height bounded by $|V|$. The transfer function $f_B(X) = GEN[B] \cup (X \setminus KILL[B])$ is monotonic ($X \subseteq Y \implies f_B(X) \subseteq f_B(Y)$). By **Tarski's Fixed Point Theorem**, iterative evaluation over a finite semi-lattice must monotonically ascend and converge to the unique Least Fixed Point (LFP) in at most $\mathcal{O}(|V| imes |E|)$ steps, with zero possibility of infinite cycling.

Q7 **Why does an assignment `x = 0` act as a KILL for variable `x` ?**

Answer: Overwriting variable `x` with a constant literal or clean value severs the data dependency between `x` and any prior untrusted input source. Therefore, `x` is added to $KILL[B]$, removing it from the outgoing taint set and preventing false positives downstream.

Q8 What is the "Maximal Munch" rule in lexical analysis, and how does Flex enforce it?

Answer: Maximal Munch (also known as the Longest Match rule) states that when multiple regular expression patterns match the current input stream, the lexer must choose the pattern that consumes the *greatest number of characters*. For example, given the input `internal_count`, the keyword rule for `int` matches 3 characters, but the generic identifier rule matches all 14 characters. Maximal Munch ensures `internal_count` is tokenized as an identifier rather than `TOK_INT` followed by an error.

Q9 What happens when two Flex rules match the exact same number of characters?

Answer: When two rules match the identical character length, Flex resolves the tie by picking the rule that appears **first in the specification file**. This is why language keywords (`int`, `if`, `while`) must always be placed *above* the generic identifier pattern (`[a-zA-Z_][a-zA-Z0-9_]*`) in `scanner.l`.

Q10 What is the time complexity of the lexical analyzer and parser?

Answer:

- **Lexer:** $\mathcal{O}(N)$ time where N is the character count. Flex compiles regex patterns into a Deterministic Finite Automaton (DFA), processing each character in $\mathcal{O}(1)$ time with zero backtracking.
- **Parser:** $\mathcal{O}(M)$ time where M is the token count. An LALR(1) parser uses a pushdown automaton with deterministic shift/reduce actions, processing each token in $\mathcal{O}(1)$ time.

Q11 What is the time and space complexity of the Kildall data-flow solver?

Answer:

- **Time Complexity:** $\mathcal{O}(|V| imes |E|)$, where $|V|$ is the number of program variables and $|E|$ is the number of control-flow edges. Because the lattice height is $|V|$, each basic block can change its outgoing state at most $|V|$ times before saturating at the fixpoint.
- **Space Complexity:** $\mathcal{O}(|B| imes |V|)$ storing the bitsets/hash sets of tainted variables for $IN, OUT, GEN,$ and $KILL$ across all $|B|$ basic blocks.

Q12 **How does CodeGuardian prevent memory leaks in the C11 compiler frontend?**

Answer: All AST allocations in `ast.c` use centralized wrappers with null-check validations. When compilation completes, the root AST node is passed to `ast_free()`, which executes a post-order recursive traversal freeing all child nodes and string allocations before exit. Similarly, TAC quadruples and basic block arrays maintain explicit capacity tracking and are systematically released.

Q13 **What vulnerabilities are detected by CodeGuardian?**

Answer:

1. **CWE-89 (SQL Injection):** Untrusted data reaching `execute_query()` without prior invocation of `escape_sql()` or `sanitize()`.
2. **CWE-78 (Command Injection):** Untrusted data reaching `system_exec()` without prior invocation of `sanitize()`.
3. **CWE-79 (Cross-Site Scripting - XSS):** Untrusted data reaching `render_output()` without prior invocation of `html_encode()`.

Q14 **How does the auditor trace the exact vulnerability path from Source to Sink?**

Answer: When a sink instruction in block \(B_{ ext{sink}} \) is detected with a tainted argument, `auditor.js` performs a backwards search through predecessor blocks. It traverses backwards along edges carrying that tainted variable in their \(OUT \) sets until reaching the block where the variable originated in \(GEN[B_{ ext{source}}] \). This sequence of block IDs (\(B_0 o \dots o B_{ ext{sink}} \)) is recorded in `taintPath`.

Q15 **How does the compiler bridge communicate with the Node.js security engine?**

Answer: Across a strongly typed, frozen JSON data contract: `CFG.json`. The frontend CLI (`codeguardian-frontend`) is invoked via process piping or subprocess execution. The emitted JSON encapsulates basic block definitions, TAC quadruples, and control-flow edges. The security engine validates this output against `cfg.schema.json` using Ajv before solving. Process isolation ensures native C segfaults cannot crash the server.

Q16 What is the difference between Synthesized and Inherited attributes in Syntax-Directed Translation?

Answer:

- **Synthesized Attributes:** Computed from the values of attributes at the child nodes in the parse tree (bottom-up flow).
- **Inherited Attributes:** Computed from the values of attributes at the parent or sibling nodes in the parse tree (top-down / lateral flow).

CodeGuardian's TAC generator uses synthesized attributes: expression quadruples are synthesized from sub-expression operands and returned up the parse tree.

Q17 Why are pointers and dynamic heap allocations excluded from Mini-C?

Answer: In standard C, pointer aliasing (\(*p = q \)) introduces undecidable points-to sets during static analysis. By restricting Mini-C to deterministic scalar types (`int` , `bool`), CodeGuardian achieves sound, complete, and mathematically verifiable data-flow taint tracking without the undecidability of unbounded memory alias sets.

Q18 What is SARIF, and why is CodeGuardian's export significant?

Answer: SARIF (Static Analysis Results Interchange Format, OASIS Standard 2.1.0) is the universal format for static analysis output. Major platforms like GitHub Advanced Security and GitLab CI ingest SARIF files directly to annotate pull requests with inline security warnings. CodeGuardian exports compliant SARIF documents with rule descriptors, physical source locations, and code snippets, making it enterprise CI/CD ready.

Q19 How does CodeGuardian guarantee that auto-remediation patches are semantically correct?

Answer: Rather than blindly stripping code, CodeGuardian inspects `security_rules.json` to determine the authorized sanitizer for the detected CWE (e.g. `escape_sql()` for CWE-89, `sanitize()` for CWE-78, `html_encode()` for CWE-79). The patch generator (`remediation.js`) identifies the tainted variable argument and synthesizes a wrapper call directly before the sink. The user can then click "Apply Fix & Re-Analyze" to verify that the fix clears the vulnerability.

Q20 How does Cytoscape.js render the Control Flow Graph in the dashboard?

Answer: The dashboard uses the `cytoscape-dagre` hierarchical directed graph layout. Basic blocks are rendered as nodes with block IDs and TAC previews. Nodes dynamically adapt their SVG backgrounds based on the active theme (Dark vs Light) and color-code blocks according to lattice audit states: Emerald Green (Clean), Cyan (Sanitized), Amber (Tainted), and Crimson Red (Vulnerable). Attack paths feature animated glowing pulses.

Q21 What was the measured performance of CodeGuardian against the PRD NFR-02 SLA?

Answer: PRD NFR-02 mandated an end-to-end analysis latency under **300 ms**. Automated stopwatch benchmarks across all canonical scenarios demonstrated an average total execution time of **~162 ms** (Frontend ~21 ms + Security Engine ~141 ms), leaving over **137 ms of safety buffer headroom** below the SLA limit.

Q22 What is the purpose of package-lock.json?

Answer: While `package.json` defines semver range dependencies (e.g. `^18.3.1`), `package-lock.json` records the exact, immutable dependency tree, exact commit hashes, and cryptographic integrity hashes (SHA-512). This guarantees that every developer and CI runner installs the exact identical dependencies, preventing build drift across machines.

Q23 What is a Basic Block leader, and why does control enter only at the leader?

Answer: A leader is the first instruction of a basic block. By definition, a basic block is a sequence of consecutive instructions where control enters solely at the first instruction and leaves solely at the last instruction. If execution could jump into the middle of a block, intermediate statements might be bypassed, invalidating the basic block's single-entry single-exit invariant and corrupting data-flow transfer equations.

Q24 How does the system fail closed when syntactically invalid code is supplied?

Answer: When invalid syntax is parsed, Bison's `yyerror()` captures the exact line and column, emits a structured error diagnostic, and terminates with exit code 1 without generating a CFG. The HTTP gateway catches this failure and returns an HTTP `422 Unprocessable Entity` response containing diagnostic line coordinates, preventing unsafe execution.

Q25 **How does CodeGuardian prove that loops with cyclic back-edges do not loop infinitely?**

Answer: Loop back-edges in Mini-C (e.g. `GOTO L_head`) re-insert the loop header basic block into Kildall's worklist queue whenever the loop body modifies \(OUT \). However, because variable taint sets strictly accumulate monotonically (\(IN_{ ext{new}} \supseteq IN_{ ext{old}} \)) and the variable set \(|V| \) is finite, a block can change its outgoing state at most \(|V| \) times. Once no further variables can be added to the taint set, \(OUT[B] \) stops changing and the worklist becomes empty, guaranteeing termination.

5. Performance Benchmarks & Gate Acceptance

CodeGuardian was subjected to rigorous end-to-end performance benchmarking across all four canonical CWE test programs using high-resolution system timers (`scripts/benchmark.ps1`). The PRD Non-Functional Requirement **NFR-02** established an end-to-end SLA threshold of **< 300 ms**.

5.1 Quantitative Latency Benchmark Table

Test Program	LOC	Blocks	Edges	Vulns	Frontend (Avg)	Engine (Avg)	Total Latency	PRD SLA Status
clean_flow.c	8	1	0	0	22.11 ms	152.71 ms	**174.82 ms**	**PASS (< 300ms)**
sql_injection.c	7	1	0	1	23.02 ms	140.96 ms	**163.99 ms**	**PASS (< 300ms)**
command_injection.c	16	4	4	1	19.32 ms	135.62 ms	**154.94 ms**	**PASS (< 300ms)**
complex_loop.c	15	4	4	1	20.81 ms	135.28 ms	**156.09 ms**	**PASS (< 300ms)**

SLA Verification Verdict

- **Target SLA Budget:** < 300 ms
- **Observed Average Latency: 162.46 ms**
- **Safety Buffer Margin: 137.54 ms of buffer headroom (45.8% under budget)**
- **Acceptance Gate 5: VERIFIED & ACCEPTED**

5.2 Milestone Acceptance Gate Review Status

Gate	Milestone Scope	Acceptance Criteria	Verification Command	Status
Gate 1	Grammar & Contract Freeze	Zero shift/reduce conflicts; valid CFG & Audit JSON schemas	`make test`	**VERIFIED (PASS)**
Gate 2	Compiler Core Lowering	AST synthesis, TAC quadruples, Dragon Book leaders, `CFG.json`	`./compiler_core/bin/codeguardian-frontend.exe`	**VERIFIED (PASS)**
Gate 3	Security Solver & Pipeline	Kildall worklist LFP convergence, 62 unit tests, CLI piping	`cd security_engine && npm test`	**VERIFIED (PASS)**
Gate 4	Visual Dashboard Integration	Cytoscape Dagre graph, line jump sync, Diff modal, SARIF export	`cd dashboard && npm run build`	**VERIFIED (PASS)**
Gate 5	Performance & SLA Review	End-to-end latency < 300 ms, multi-platform verification runner	`powershell -File scripts/verify_all.ps1`	**VERIFIED (PASS)**

End of CodeGuardian System Handbook · Generated for Academic Viva Defense & Engineering Evaluation · All 5 Acceptance Gates Verified